

Sesiunea 3: JavaScript

pentru QA Automation

Capitolul 1: Bazele programării în JS. Variabile, Tipuri de date, Structuri de control și Funcții	2
Obiective de învățare	2
1.1 Marea trecere: De ce învățăm JavaScript?	3
1.2 Mediul de lucru: Unde scriem și rulăm codul JS?	3
1.3 Variabile: Cutiile cu Memorie (let vs const)	4
1.4 Tipuri de Date: Ce punem în cutii?	6
1.5 Structuri de Control: Inteligența Scriptului	6
1.6 Funcții: Rețetele reutilizabile.....	7
1.7 Știați că...?.....	8
1.8 Poveste Aplicată: "Bucătăria JavaScript"	8
1.8 Exerciții Practice	9
1.9 Soluții la Exerciții.....	9
1.10 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață	10
Capitolul 2: Ce punem în cutii? Tipurile de date fundamentale.....	11
Obiective de învățare	11
2.1 Tipuri Primitive: Ingredientele de bază.....	12
.....	13
2.2 Misterul "Nimicului": undefined vs null	14
2.3 Structuri Complexe: Array-uri și Obiecte	14
2.4 Știați că...?	15
2.5 Exerciții Practice.....	15
2.6 Soluții la Exerciții	15
2.7 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață	16
Capitolul 3: Logica aplicației - Structuri de Control (if/else, for)	18
Obiective de învățare	18
3.1 Luarea Deciziilor: Instrucțiunea if / else	18
3.2 Mai multe ramuri: else if.....	20

3.3 Repetiția: Bucla for (The for Loop)	20
3.4 Iterarea prin Liste (Combinarea Buclelor cu Array-urile)	21
3.5 Știați că...?	21
3.6 Exerciții Practice.....	22
3.7 Soluții la Exerciții	22
3.8 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață.....	23
Capitolul 4: Reutilizarea codului - Funcții clasice și Funcții Săgeată (Arrow Functions)	24
Obiective de învățare	24
4.1 Ce este o funcție și de ce avem nevoie de ea?	24
4.2 Funcția Clasică	25
4.3 Funcția Săgeată (Arrow Function) - Standardul Modern.....	26
4.4 Știați că...?	27
4.5 Exerciții Practice	27
4.6 Soluții la Exerciții.....	28
4.7 🤖 Mentalitatea de Inginer în Era AI: Locul tău pe piață.....	28
Capitolul 5: Tema Sesiunii 3 & Extra Practice (Nivel Avansat).....	30
5.1 Tema pentru Acasă (Proiect Sesiunea 3)	30
5.2 Provocarea Bonus (Opțional): Capcana "Falsy" a API-urilor	33
5.3 🗝️ Cheile de Rezolvare (Soluții & Explicații)	33

Capitolul 1: Bazele programării în JS.

Variabile, Tipuri de date, Structuri de control și Funcții

Obiective de învățare

🧠 La finalul acestui capitol, vei fi capabil să:

- Înțelegi rolul JavaScript-ului (JS) în testarea automatizată (Playwright) și să cunoști mediile în care îl poți executa (IDE vs. Browser).
- Declari variabile folosind `let` și `const` și să înțelegi diferența vitală dintre ele.
- Identifici și utilizezi tipurile de date primitive (String, Number, Boolean).
- Controlezi fluxul execuției folosind instrucțiuni decizionale (`if/else`) și bucle (`for`).
- Scrii blocuri de cod reutilizabile folosind funcții clasice și funcții săgeată (Arrow Functions).

1.1 Marea trecere: De ce învățăm JavaScript?

Dacă HTML-ul este "scheletul" casei, iar CSS-ul "designul", **JavaScript (JS)** reprezintă "sistemul electric și instalația de apă". JS transformă o pagină web statică într-o aplicație interactivă.

Pentru tine, ca viitor QA Automation Engineer, JavaScript este *limba în care vei vorbi cu robotul tău*. Playwright și Cypress sunt framework-uri construite pe bază de JavaScript (și extensia sa, TypeScript). Nu poți scrie scenarii de testare complexe, nu poți valida rezultate și nu poți manipula date fără a stăpâni elementele de bază ale acestui limbaj.

1.2 Mediul de lucru: Unde scriem și rulăm codul JS?

Limbajul JavaScript a fost creat inițial doar pentru a rula în browser, dar astăzi rulează oriunde. Pentru a testa exemplele din acest curs, vei folosi două medii principale:

1. Consola Browserului (DevTools) - "Terenul de joacă rapid"

Cel mai rapid mod de a rula cod JS este chiar browserul tău Chrome.

- *Cum accesezi:* Apeși F12 (sau click-dreapta -> *Inspect*) și mergi pe tab-ul **Console**. Acolo poți scrie direct cod JavaScript, apeși Enter și vezi rezultatul instantaneu. Este perfect pentru a testa funcții mici, a verifica logica sau a depista selectoare pe loc.

2. IDE-ul (Visual Studio Code) + Node.js - "Mediul Profesionalist"

Când scrii un proiect real de automatizare, nu poți scrie sute de linii de cod direct în browser. Vei folosi un **IDE (Integrated Development Environment)**, precum *Visual Studio Code* (pe care l-ai folosit la HTML). Pentru ca PC-ul tău să înțeleagă JS în afara browserului, în modulele următoare vom instala și folosi un motor numit **Node.js**. În VS Code, vei scrie fișiere cu extensia `.js` (sau `.ts` pentru TypeScript) și le vei rula direct dintr-un terminal integrat.

1.3 Variabile: Cutiile cu Memorie (let vs const)

O variabilă este ca o cutie de pantofi goală pe care pui o etichetă (numele). În ea stochezi o informație (valoarea) pentru a o refolosi mai târziu. În JavaScript-ul modern, avem două moduri principale de a crea aceste cutii:

1. const (Constanta - Regele automatizării)

Se folosește atunci când valoarea din "cutie" **NU** trebuie să se mai schimbe niciodată pe parcursul rulării scriptului.

- *Regula de aur:* În 90% din cazuri, în testele Playwright, vei folosi const.

```
const urlPlatforma = "https://certify.qualiadept.eu";
const butonLogin = page.locator('#login-btn');
// Dacă încerci mai târziu să scrii urlPlatforma = "google.com", codul va
"crăpa" cu eroare, protejându-te de greșeli accidentale!
```

2. let (Variabila schimbătoare)

Se folosește DOAR când știi sigur că valoarea se va modifica pe parcursul execuției (ex: un contor).

```
let numarIncerari = 0;
numarIncerari = 1; // Permis. Valoarea s-a actualizat.
```

*(Notă: S-ar putea să vezi în tutoriale foarte vechi pe internet cuvântul var. **Evită-l!** Este perimat, "scapă" din structurile de control și cauzează bug-uri grave de memorie în aplicațiile moderne).*

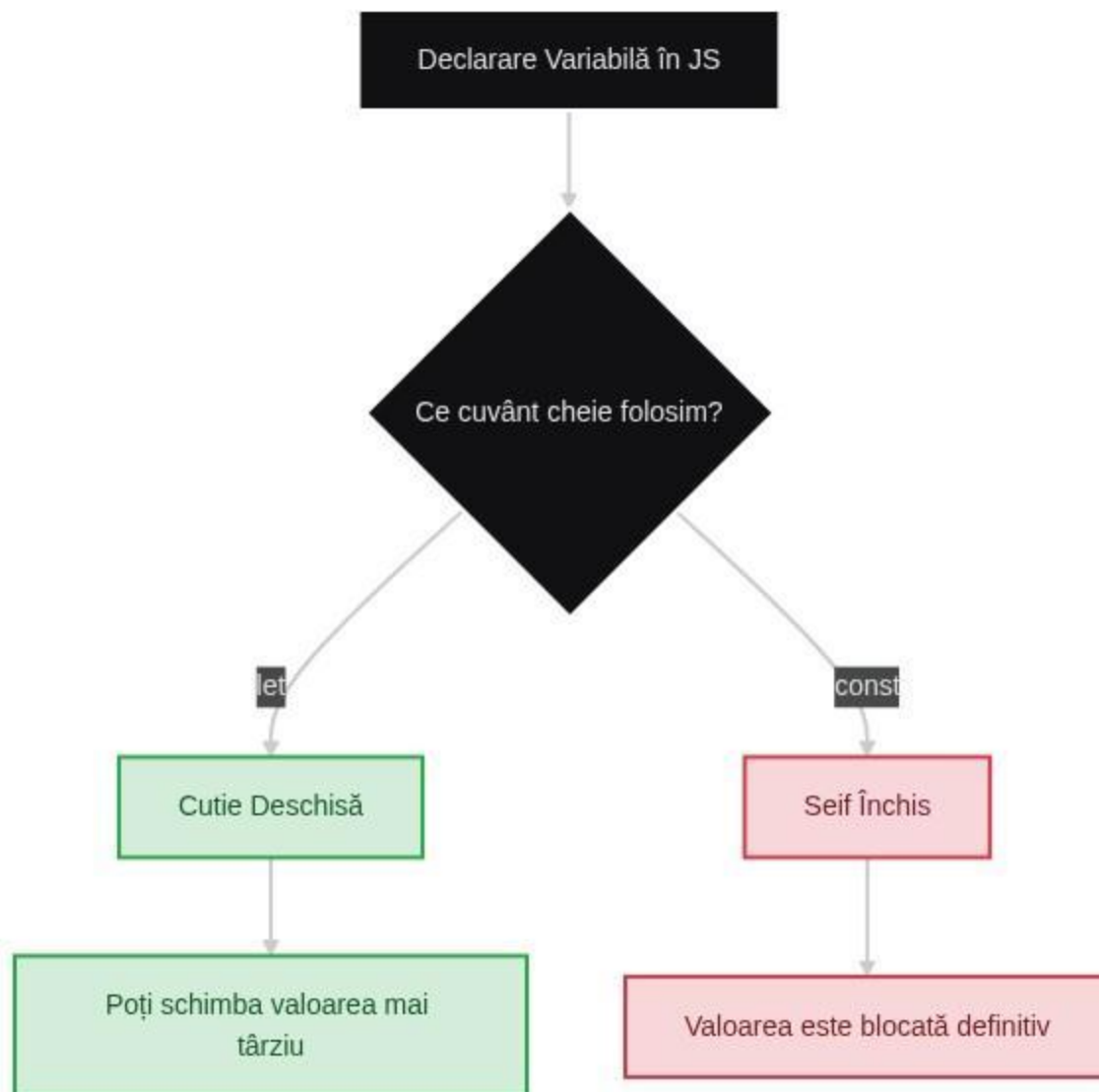


Fig. 1 - Diagrama care ilustrează fluxul de decizie la **declararea unei variabile în JavaScript**, structurat pe trei niveluri:

- **Punctul de pornire (Nivelul 1):** Un bloc central intitulat „Declarare Variabilă în JS”.
- **Întrebarea de control (Nivelul 2):** O casetă de decizie tip romb care pune întrebarea: „Ce cuvânt cheie folosim?”. Din acest punct, fluxul se împarte pe două ramuri:
 - **Ramura din stânga (let):** Conduce către o casetă verde deschis numită „Cutie Deschisă”, urmată de explicația finală: „Poți schimba valoarea mai

târziu”.

- **Ramura din dreapta (const):** Conduce către o casetă roșu deschis numită „Seif Închis”, urmată de explicația finală: „Valoarea este blocată definitiv”.

1.4 Tipuri de Date: Ce punem în cutii?

Pentru ca robotul să ne înțeleagă, trebuie să știe exact cu ce fel de date lucrează. Tipurile fundamentale sunt:

- **String (Șir de caractere/Text):** Se scrie mereu între ghilimele (simple '...', duble "...", sau backticks `...`).
 - Exemplu QA:

```
const username = "testuser_1";
```

- **Number (Numere):** Se scriu liber, fără ghilimele.
 - Exemplu QA:

```
const timeoutTest = 5000; (Robotul va aștepta 5000 de milisecunde).
```

- **Boolean (Adevărat/Fals):** Are doar două valori posibile: true sau false.
 - Exemplu QA:

```
const esteVizibil = true;
```

(Util pentru validări: Aștept ca butonul să fie vizibil -> true).

1.5 Structuri de Control: Inteligența Scriptului

Un script inteligent nu rulează orbește de sus în jos, ci ia decizii și repetă acțiuni.

1. Luarea deciziilor (if / else)

Permite testului tău să reacționeze diferit în funcție de situație.

```
let mesajEroareVizibil = true;

if (mesajEroareVizibil === true) {
  console.log("Testul a picat, a apărut o eroare pe ecran!");
}
```

```

} else {
    console.log("Testul continuă, totul este în regulă.");
}

```

2. Repetiția (Bucla for)

Ai o listă cu 10 produse în coș și vrei să verifici că fiecare are prețul afișat. Nu scrii codul de 10 ori! Folosești o buclă for:

```

// Structură: for(Start; Condiție de oprire; Pasul)
for (let i = 1; i <= 3; i++) {
    console.log("Verific produsul numărul: " + i);
}
// Va afișa: 1, apoi 2, apoi 3.

```

1.6 Funcții: Rețetele reutilizabile

O funcție este un bloc de cod cu un nume, care execută o acțiune specifică. În loc să scrii de 100 de ori pașii pentru "Autentificare" în fiecare test, faci o funcție pe care o apelezi oriunde ai nevoie!

1. Funcția Clasică:

```

function adunaNumere(a, b) {
    return a + b;
}
const rezultat = adunaNumere(5, 10); // rezultat devine 15

```

2. Funcția Săgeată (Arrow Function - Standardul Modern):

Aceasta este sintaxa pe care o vei vedea peste tot în Playwright. Este mai scurtă și mai curată. Scoatem cuvântul function și punem o "săgeată" => după parametri.

```

const adunaNumere = (a, b) => {
    return a + b;
};

// Exemplu de automatizare (simplificat):
const clickPeButon = (selector) => {

```

```
console.log("Playwright va da click pe: " + selector);  
};  
clickPeButon("#login-btn");
```

1.7 Știați că...?

💡 Știați că...

- **JavaScript NU este Java:** Limbajul **JavaScript** a fost creat în anul **1995** de către **Brendan Eich** în doar **10** zile! Inițial s-a numit **Mocha**, apoi **LiveScript**. Numele final de **"JavaScript"** a fost doar o manevră de marketing pentru a profita de popularitatea uriașă pe care o avea limbajul **"Java"** în acea perioadă. După cum se spune în industrie: „**Java și JavaScript se aseamănă la fel de mult precum ham (șuncă) și hamster**”.
- **Camel Case:** Standardul de scriere a numelor de variabile în JS se numește *camelCase* (ca o cocoasă de cămilă). Primul cuvânt e mereu cu literă mică, iar următoarele încep cu literă mare, lipite, fără spații (ex: numeUtilizatorNou, butonTrimiteFormular).
- **De la animații la servere și roboți:** Dacă inițial JS a fost inventat doar pentru a face o zăpadă să cadă pe ecran sau un buton să clipească în browser, astăzi, datorită platformei *Node.js*, poți rula servere complexe, poți programa roboți fizici și poți scrie sisteme de testare enterprise, totul cu un singur limbaj!

1.8 Poveste Aplicată: "Bucătăria JavaScript"

🧑‍🍳 Gândește-te la bucătăria ta:

- **IDE-ul (VS Code)** este bucătăria însăși, unde ai toate instrumentele și unde prepari rețetele complexe. **DevTools Console** este cuptorul cu microunde: rapid, dar doar pentru teste/încălziri mici.
- **Variabilele (const și let)** sunt recipientele tale. Borcanul cu sare este un **const** (rămâne mereu sare, nu se schimbă). Bolul de salată este un **let** (azi faci salată de roșii, mâine îl

speli și pui fructe).

- **Tipurile de date** sunt ingredientele (String = Făină, Number = 3 ouă).
- **Structurile de control (if/else)** sunt deciziile tale ca bucătar: *"Dacă supa e prea nesărată, mai pun sare. Altfel, o opresc."*
- **Funcția** este Rețeta. Ai o rețetă scrisă (funcția), îi dai ingredientele (parametrii) și ea îți returnează preparatul (return).

1.8 Exerciții Practice

Pentru a rezolva aceste exerciții, deschide un tab nou în Chrome, apasă F12, mergi la tab-ul **Console** și scrie codul acolo, apăsând Enter după fiecare linie!

Exercițiul 1: Declararea corectă

- Declară o constantă numită numeAplicatie și dă-i valoarea "Task Tracker".
- Declară o variabilă (care se poate schimba) numită testeTrecute și inițializeaz-o cu valoarea 0.
- Încearcă să schimbi valoarea constantei numeAplicatie = "Alta Aplicație". Ce eroare îți afișează consola?

Exercițiul 2: Arrow Function pentru Vârstă

Scrie o **funcție săgeată (arrow function)** numită verificaMajorat care primește un parametru numit varsta (un Number). Funcția trebuie să aibă un if/else:

- Dacă vârsta este mai mare sau egală ($\geq$) cu 18, să dea return "Utilizator major".
- Dacă nu, să dea return "Acces interzis".

Apelează funcția în consolă cu vârsta ta pentru a o testa!

1.9 Soluții la Exerciții

✓ Soluție Exercițiul 1:

```
const numeAplicatie = "Task Tracker";
let testeTrecute = 0;
// Eroarea la modificarea constantei va fi: Uncaught TypeError: Assignment
```

to constant variable.

✓ Soluție Exercițiul 2:

```
const verificaMajorat = (varsta) => {
  if (varsta >= 18) {
    return "Utilizator major";
  } else {
    return "Acces interzis";
  }
};

console.log(verificaMajorat(20)); // Afișează: Utilizator major
console.log(verificaMajorat(16)); // Afișează: Acces interzis
```

1.10 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe piață

🔍 QA vs. AI: Iluzia codului perfect generat

Odată cu apariția AI-ului, mulți cred că "nu mai trebuie să învăț programare, îmi scrie AI-ul codul". Aceasta este o capcană uriașă pentru un QA.

- **Ce face AI-ul bine:** Dacă îi ceri "scrie-mi o funcție JS care validează formatul unui email", AI-ul îți va da în 2 secunde un cod complex pe care probabil ți-ar fi luat 20 de minute să-l scrii de la zero. E un asistent grozav.
- **Capcana (Unde AI-ul eșuează):** AI-ul este predispus la "halucinații logice" și la practici slabe de arhitectură dacă nu primește contextul perfect. El ar putea scrie un întreg scenariu de test folosind doar let în loc de const, expunând datele testului tău la modificări accidentale (mutation bugs). Sau ar putea genera o buclă for infinită care îți va bloca complet serverul sau browser-ul.
- **Cum gândește un inginer QA:** Inginerul citește codul generat de AI la fel cum un profesor corectează teza unui elev. El va spune: "Mulțumesc pentru scheletul logic, dar ai folosit var în loc de let, și logica ta de if/else nu acoperă scenariul în care formularul

este complet gol." Inginerul QA va corecta și refactoriza funcția generată în IDE-ul său pentru a o face robustă (bulletproof).

- **De ce vei rămâne relevant pe piață:** Nu te bat roboții care scriu cod, te bat oamenii care știu ce să le ceară roboților și cum să le valideze rezultatul. Cunoașterea profundă a fundamentelor (Memorie, Bucle, Logica Decizională) te face Arhitectul sistemului de testare, în timp ce AI-ul rămâne doar un executant rapid. Valoarea ta reală stă în validarea logicii de business și asigurarea trasabilității testelor.

Capitolul 2: Ce punem în cutii? Tipurile de date fundamentale

Obiective de învățare



- Identifici și utilizezi tipurile de date primitive (String, Number, Boolean).
- Înțelegi diferența vitală dintre undefined și null.
- Lucrezi cu tipuri de date complexe: Array-uri (liste) și Obiecte (colecții).
- Investigați datele folosind operatorul typeof.

2.1 Tipuri Primitive: Ingredientele de bază

Dacă variabilele (let și const) sunt "cutiile", tipurile de date ne spun ce fel de obiecte se află în acele cutii. În JavaScript, ca în orice limbaj, avem câteva tipuri simple, numite "primitive".

- **String (Text):** Orice șir de caractere. În JS, textul trebuie **întotdeauna** îmbrăcat în ghilimele (simple '...', duble "...", sau backticks `...`).

```
const mesajEroare = "Parola este incorectă!";
```

- **Number (Numere):** Spre deosebire de alte limbaje care fac diferența între numere întregi și zecimale, JS le numește pe toate simplu, Number. Nu se pun în ghilimele!

```
const timpAsteptare = 5000; // 5 secunde în milisecunde
const pretProdus = 99.99;
```

- **Boolean (Adevărat/Fals):** Are doar două valori posibile: true sau false (fără ghilimele).

Este fundamental pentru logica decizională.

```
let esteVizibil = true;
let utilizatorLogat = false;
```

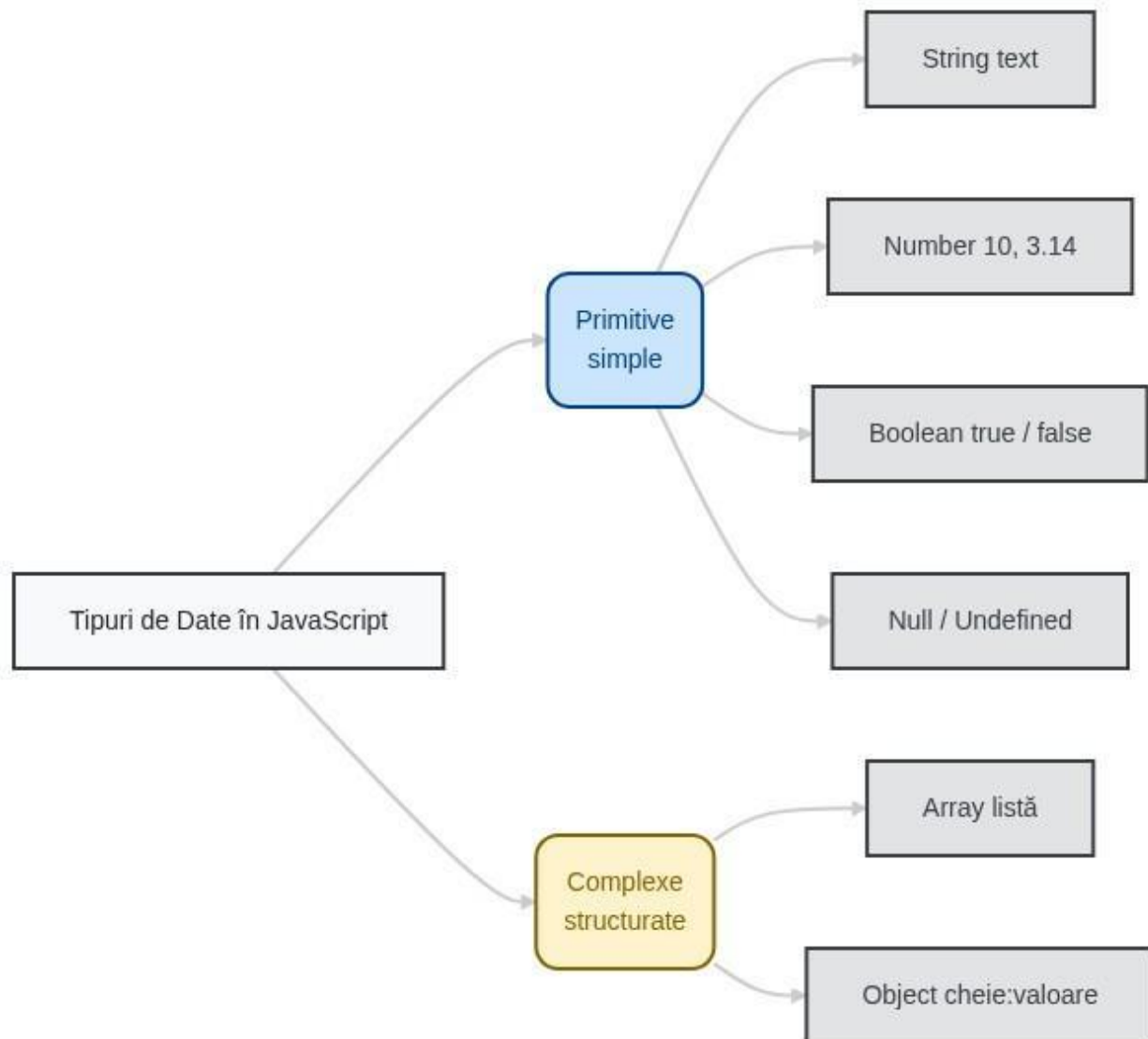


Fig. 2 - Diagrama prezintă clasificarea **Tipurilor de Date în JavaScript**, structurată pe două ramuri principale:

- **Tipurile Primitive (simple):**
 Reprezentate într-un bloc albastru. Acestea includ:
 - **String** (text)
 - **Number** (numere precum 10 sau 3.14)
 - **Boolean** (valori de adevăr: true / false)
 - **Null / Undefined** (lipsa unei valori)
- **Tipurile Complex (structurate):** Reprezentate într-un bloc galben. Acestea

includ:

- **Array** (liste de date)

2.2 Misterul "Nimicului": undefined vs null

Ca QA, vei întâlni des aceste două concepte când robotul caută ceva pe pagină și nu găsește.

- **undefined (Nedefinit):** Înseamnă că ai creat cutia, dar *încă nu ai pus nimic în ea*. Calculatorul nu știe ce este.

```
let elementCautat;
console.log(elementCautat); // Afișează: undefined
```

- **null (Nul):** Înseamnă "nimic", dar este un nimic *intenționat*. Tu îi spui explicit calculatorului: "Această cutie este goală pe moment".

```
let dateUtilizator = null; // Vom pune datele aici după ce se loghează
```

2.3 Structuri Complexe: Array-uri și Obiecte

Adesea, nu vrem să lucrăm cu o singură valoare, ci cu o colecție de date.

1. Array-ul (Lista): O listă ordonată de valori, scrisă între paranteze drepte []. Gândește-te la o cutie compartimentată. Fiecare element are un index (o poziție), dar atenție: **numărătoarea începe mereu de la 0!**

```
const browsereSuportate = ["Chrome", "Firefox", "Webkit"];
// "Chrome" este la poziția 0, "Firefox" la poziția 1
console.log(browsereSuportate[0]); // Afișează: Chrome
```

2. Obiectul (Object): Folosit pentru a descrie o "entitate" cu multiple caracteristici. Se scrie între acolade { } și conține perechi *cheie: valoare*. Este formatul standard (JSON) în care aplicațiile web comunică între ele (API-uri).

```
const utilizatorTest = {
  username: "qa_expert",
  parola: "Test1234!",
  esteAdmin: true
};
// Pentru a accesa o proprietate, folosim punctul (.):
console.log(utilizatorTest.username); // Afișează: qa_expert
```

2.4 Știați că...?

- **Detectorul de tipuri:** JavaScript are un cuvânt magic numit `typeof` pe care îl poți folosi în Consolă ca să afli ce se ascunde într-o variabilă. Dacă scrii `typeof 42`, îți va răspunde "number". Dacă scrii `typeof "42"`, îți va răspunde "string".
- **Totul e un obiect:** Sub capotă, în JavaScript, inclusiv Array-urile sunt considerate o formă specială de "Obiect" cu chei numerice!
- **Atenție la adunări:** Ce se întâmplă dacă aduni un Number cu un String? `5 + "5"` în JavaScript nu face 10, ci "55". JS transformă numărul în text și le lipește! Acesta este motivul pentru care tipurile de date sunt critice.

2.5 Exerciții Practice

Deschide VS Code (creează un fișier `test.js` și rulează-l cu Node.js, dacă ai instalat mediul) sau folosește tab-ul **Console** din DevTools (Chrome).

Misiunea ta:

- Declară o variabilă `varsta` (Number) cu valoarea ta.
- Declară un Obiect numit `produsTest` care să aibă 3 caracteristici: `nume` (String), `pret` (Number) și `inStoc` (Boolean).
- Folosește comanda `console.log()` pentru a afișa prețul produsului (folosind sintaxa cu punct).
- Rulează comanda `typeof produsTest.inStoc` și vezi ce îți spune consola.

2.6 Soluții la Exerciții

```
let varsta = 28;

const produsTest = {
  nume: "Căști Wireless",
  pret: 299.99,
  inStoc: true
};

console.log(produsTest.pret); // Va afișa 299.99
console.log(typeof produsTest.inStoc); // Va afișa "boolean"
```

2.7 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe piață

🔍 QA vs. AI: Confuzia Tipurilor de Date (Type Coercion)

- **Ce face AI-ul bine:** Dacă îi ceri unui asistent AI *"generează-mi un profil de utilizator pentru un test"*, va scuipa imediat un obiect complex, structurat perfect vizual, cu zeci de proprietăți în format corect.
- **Capcana (Unde AI-ul eșuează):** În JavaScript există o mare capcană: `==` (egalitate vagă) vs `===` (egalitate strictă). Un AI care nu înțelege logica exactă de business a aplicației tale ar putea genera un test care verifică dacă `0 == false` (ceea ce în JS este `true`, deoarece le forțează să se transforme). Sau ar putea trimite către server (via API) un preț de formă `"100"` (String) în loc de `100` (Number). Serverul va da crash (eroare 500), iar AI-ul se va învârti în cerc neștiind de ce.
- **Cum gândește un inginer QA:** Inginerul este obsedat de "Tipuri Stricte". El se uită pe scriptul generat de AI și observă că aserțiunea (validarea) este fragilă. Inginerul va corecta testul pentru a folosi **mereu** `===` (care verifică atât valoarea, CÂT ȘI tipul de date). El știe că un String nu are voie niciodată să treacă drept Number într-un sistem financiar.
- **De ce vei rămâne relevant pe piață:** Valoarea ta majoră în echipa de dezvoltare nu este doar "să dai click pe butoane automat", ci **Asigurarea Calității Datelor**. Tu previi

acele bug-uri invizibile, " silențioase", pe care un AI le introduce accidental din cauză că nu "înțelege" diferența fundamentală dintre un șir de caractere care reprezintă o vârstă și un număr matematic cu care aplicația trebuie să calculeze taxe. Tu aduci Rigoare într-un limbaj adesea prea permisiv!

Capitolul 3: Logica aplicației -

Structuri de Control (if/else, for)

Obiective de învățare



- Înțelegi cum se execută codul (fluxul de control de sus în jos) și cum îl poți devia.
- Scrii instrucțiuni decizionale (if / else if / else) pentru a trata diferite scenarii.
- Utilizezi bucla for pentru a repeta automat blocuri de cod.
- Combini Array-urile (din capitolul precedent) cu buclele for pentru a itera prin liste de date.

3.1 Luarea Deciziilor: Instrucțiunea if / else

În mod normal, JavaScript citește codul tău exact cum citești tu o carte: linie cu linie, de sus în jos. Structurile de control ne permit să schimbăm această ordine.

Cea mai simplă și folosită decizie este instrucțiunea **dacă / altfel (if / else)**. Îi spune calculatorului: *"Dacă această condiție este adevărată, execută blocul A. Altfel, execută blocul B."*

```
let elementVizibil = true;

if (elementVizibil === true) {
    // Acest cod se execută DOAR dacă condiția din paranteze este adevărată (true)
    console.log("Click pe buton!");
} else {
    // Acest cod se execută dacă condiția este falsă (false)
    console.log("Așteaptă ca elementul să apară...");
}
```

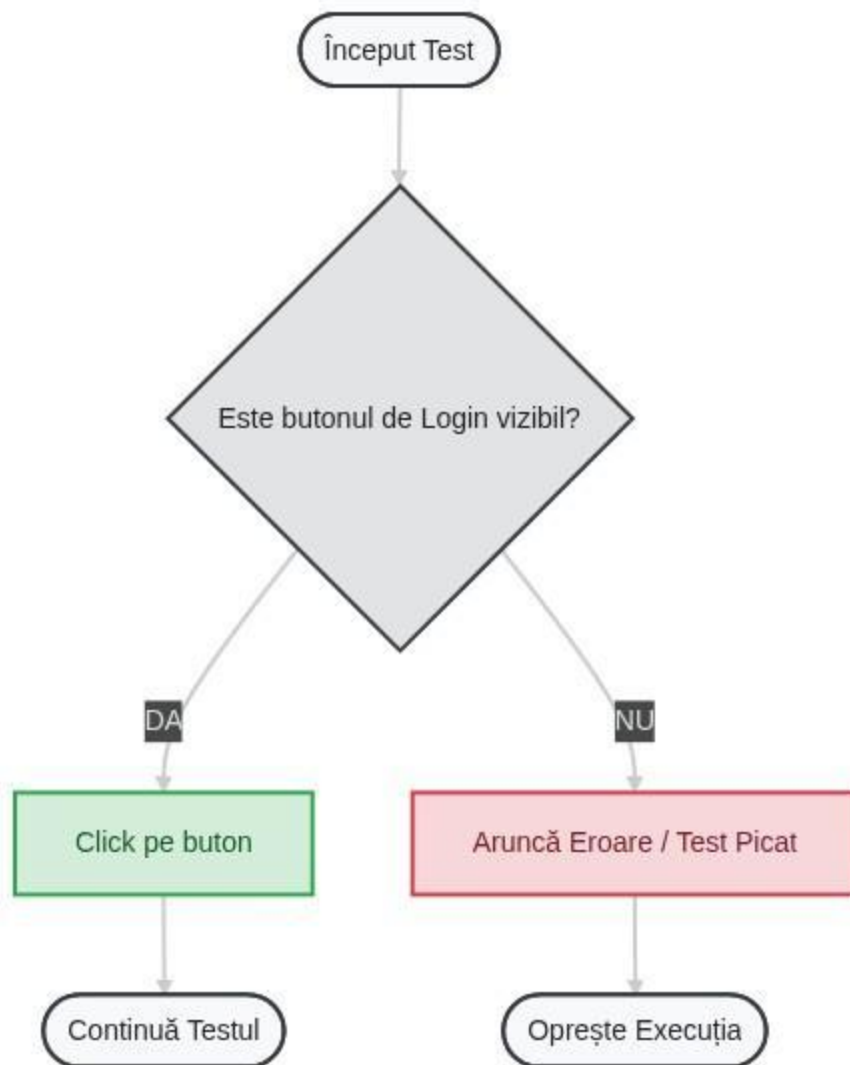


Fig. 3 - Diagrama ilustrează execuția condiționată (**Logica if/else**) într-un script de testare automată:

- **Punctul de pornire:** Testul începe cu verificarea unei condiții într-un bloc de decizie: „Este butonul de Login vizibil?”.
- **Ramura if (Adevărat / DA):** Dacă butonul este vizibil, codul execută acțiunea Click pe buton (marcată cu verde), iar apoi fluxul merge spre finalul pozitiv: Continuă Testul.
- **Ramura else (Fals / NU):** Dacă butonul nu este vizibil, codul deviază către acțiunea de eroare Aruncă Eroare / Test Picat (marcată cu roșu), iar execuția se oprește definitiv prin Oprește Execuția.

3.2 Mai multe ramuri: else if

În viața reală de QA, nu avem doar alb și negru. Ce se întâmplă când testăm un API și vrem să verificăm răspunsul (Status Code)? Avem nevoie de mai multe opțiuni, folosind else if:

```
const statusCode = 404;

if (statusCode === 200) {
  console.log("Test Passed: Pagina s-a încărcat corect.");
} else if (statusCode === 404) {
  console.log("Test Failed: Pagina nu a fost găsită (Eroare de client).");
} else if (statusCode === 500) {
  console.log("Test Failed: Serverul a picat complet!");
} else {
  console.log("Eroare necunoscută.");
}

// Deoarece statusCode este 404, consola va printa al doilea mesaj.
```

3.3 Repetiția: Bucla for (The for Loop)

Unul dintre principiile de bază în programare este **DRY (Don't Repeat Yourself - Nu te repeta)**. Dacă vrei să dai click pe 5 butoane identice, nu scrii comanda de click de 5 ori. Folosești o "bucă" pentru a automatiza repetiția.

Structura buclei for are 3 părți în interiorul parantezelor:

- **Start (Inițializarea):** De unde începem să numărăm? (ex: let i = 1)
- **Condiția de oprire:** Până când repetăm? (ex: i <= 5)
- **Pasul (Incrementarea):** Cu cât creștem contorul la finalul fiecărui tur? (ex: i++ înseamnă că îl creștem cu 1).

```
for (let i = 1; i <= 3; i++) {
  console.log("Execut acțiunea numărul: " + i);
}

// Output în Consolă:
```

```
// Execut acțiunea numărul: 1
// Execut acțiunea numărul: 2
// Execut acțiunea numărul: 3
```

3.4 Iterarea prin Liste (Combinarea Buclelor cu Array-urile)

Aici devine totul extrem de relevant pentru un Automation Engineer. Imaginează-ți că ai un Array cu nume de utilizatori și vrei să testezi login-ul pentru fiecare în parte. Vei folosi o buclă for care să treacă prin fiecare element, folosind proprietatea `.length` (care ne spune câte elemente are lista).

```
const utilizatoriTest = ["admin", "editor", "cititor"];

// i pornește de la 0 pentru că indexul Array-urilor începe de la 0!
for (let i = 0; i < utilizatoriTest.length; i++) {
    console.log("Testăm login pentru utilizatorul: " + utilizatoriTest[i]);
}
```

3.5 Știați că...?



- **Bucula Infinită (Coșmarul Programatorului):** Dacă greșești "Condiția de oprire" într-un for, bucla nu se va termina niciodată. Calculatorul va repeta acțiunea la infinit până i se epuizează memoria și dă crash (îngheață). De exemplu, dacă uiți să scrii `i++`, variabila i rămâne mereu 1, iar condiția `1 <= 5` va fi mereu adevărată.
- **Truthiness & Falsiness:** În JS, o condiție de if nu are nevoie strict de true sau false. Anumite valori sunt evaluate direct ca fiind false: 0, "" (un string gol), null și undefined. Astfel, `if ("") { ... }` nu se va executa niciodată.

3.6 Exerciții Practice

Deschide DevTools (tab-ul Console) în browser sau folosește un fișier în VS Code pentru a rezolva următoarele exerciții:

Misiunea 1: Verificarea Parolei

- Declară o variabilă parolaIncorecta și asignează-i valoarea true.
- Scrie un bloc if/else. Dacă parolaIncorecta este true, afișează "Afișează mesaj de eroare UI". Altfel, afișează "Redirecționează către Dashboard".

Misiunea 2: Căutarea în tabel

- Scrie o buclă for care să numere de la 1 la 4.
- În interiorul buclei, folosește console.log() pentru a printa: "Verific rândul numărul X" (unde X este contorul tău).

3.7 Soluții la Exerciții

☒ Soluție Misiunea 1:

```
let parolaIncorecta = true;

if (parolaIncorecta === true) {
    console.log("Afișează mesaj de eroare UI");
} else {
    console.log("Redirecționează către Dashboard");
}
```

☒ Soluție Misiunea 2:

```
for (let i = 1; i <= 4; i++) {
    console.log("Verific rândul numărul " + i);
}
```

3.8 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe piață

🔊 QA vs. AI: Bucle infinite și logica oarbă la cazuri limită (Edge Cases)

- **Ce face AI-ul bine:** Generatoarele de cod sunt fantastice la a scrie rapid sintaxa plictisitoare pentru structuri repetitive. Dacă îi spui *"parcurge un array de produse și dă click"*, îți va da imediat o buclă for corectă sintactic.
- **Capcana (Unde AI-ul eșuează):** AI-ul suferă adesea de "miopie de context". El scrie codul ideal (Happy Path). Dar ce se întâmplă dacă array-ul de produse vine gol de la server? AI-ul ar putea genera o logică de if / else care presupune că produsul există mereu. Când rulezi testul în Pipeline (CI/CD) și rețeaua e lentă, array-ul e gol, testul pică, iar AI-ul îți sugerează să pui pauze artificiale (sleeps/timeouts).
- **Cum gândește un inginer:** Un inginer QA aplică *Analiza Valorilor de Frontieră (Boundary Value Analysis)* direct pe cod. Se uită la bucla for generată de AI și adaugă o validare "defensivă" înainte de ea: `if (produse.length === 0) { console.log("Test eșuat: Nu s-au încărcat produsele"); return; }`. Inginerul nu lasă bucla să pornească orbește pe date invalide.
- **De ce vei rămâne relevant pe piață:** Valoarea ta nu stă în a cunoaște sintaxa unui for pe de rost, ci în **Gândirea Critică**. AI-ul codifică algoritmul; tu ești cel care întreabă: *"Ce se întâmplă dacă condiția asta e negativă? Ce se întâmplă dacă primesc null în loc de array?"*. Tu creezi arhitectura de testare robustă pe care o mașină pur și simplu nu o poate planifica în avans.

Capitolul 4: Reutilizarea codului - Funcții clasice și Funcții Săgeată (Arrow Functions)

Obiective de învățare



- Înțelegi ce este o funcție și cum te ajută să respecti principiul DRY (Don't Repeat Yourself).
- Declari și apelezi funcții clasice folosind parametri și cuvântul cheie return.
- Scrii funcții moderne folosind sintaxa Arrow Function (=>), standardul absolut de bază în Playwright.
- Faci tranziția de la cod "monolitic" (scris la grămadă) la cod modular și curat.

4.1 Ce este o funcție și de ce avem nevoie de ea?

Imaginează-ți că scrii un test automatizat pentru un magazin online. Trebuie să te autentifici (Login) la începutul testului pentru adăugarea în coș, la începutul testului pentru finalizarea comenzii și la începutul testului pentru verificarea istoricului de cont.

Vei scrie aceiași 5 pași de cod (introducere email, parolă, click pe buton) de zeci de ori? Nu!

O **funcție** este un bloc de cod reutilizabil, care grupează mai multe instrucțiuni sub un singur nume. O scrii o singură dată și o "strigi" (apelezi) de câte ori ai nevoie, de oriunde din aplicația ta.

Analogia QA: Funcția este o rețetă de bucătărie.

- **Parametrii (Ingredientele):** Ceea ce îi dai funcției la început ca să aibă cu ce să lucreze (ex: user și parolă).

- **Corpul funcției (Gătitul):** Codul propriu-zis care execută pașii logici în interior.
- **Return (Preparatul final):** Rezultatul procesat pe care funcția ți-l dă înapoi după ce a terminat (ex: un semnal că te-ai logat cu succes).

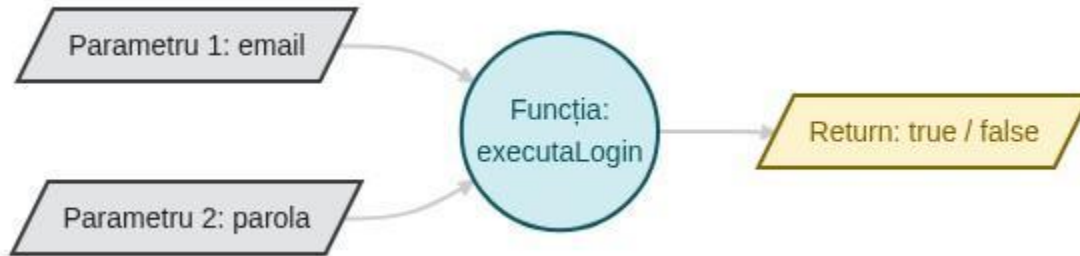


Fig. 4 - Diagrama explică conceptual **Anatomia unei Funcții** în JavaScript, folosind modelul de intrări și ieșiri:

- **Intrările (Parametrii):** În partea stângă, două casete paralele reprezintă datele de intrare transmise funcției: „*Parametru 1: email*” și „*Parametru 2: parola*”.
- **Procesarea (Funcția):** Săgețile conduc către un bloc circular central albastru, intitulat „*Funcția: executaLogin*”. Acesta acționează ca o „cutie neagră” în care se rulează logica de verificare.
- **Ieșirea (Rezultatul):** Din funcție pornește o săgeată către o casetă galbenă din dreapta, care reprezintă rezultatul returnat: „*Return: true / false*”.

4.2 Funcția Clasică

Sintaxa tradițională în JavaScript (cea cu care a pornit limbajul) folosește cuvântul cheie `function`.

```

// Declararea funcției
function calculeazaPretTotal(pretBaza, taxa) {
    let pretFinal = pretBaza + taxa;
    return pretFinal; // Returnează valoarea calculată înapoi către cine a
    apelat funcția
}
  
```

```
// Apelarea (folosirea) funcției
let totalCos = calculeazaPretTotal(100, 19);
console.log("Totalul de plată este: " + totalCos); // Afișează în consolă:
119
```

Observă că **pretBaza** și taxa sunt niște variabile temporare (parametri) care "prind" valorile 100 și 19 abia în momentul în care funcția este executată pe linia cu apelarea.

4.3 Funcția Săgeată (Arrow Function) - Standardul Modern

Deși funcțiile clasice sunt încă perfect valide și folosite, în 2015 JavaScript a introdus o sintaxă mult mai scurtă și mai curată: **Arrow Function** ($\Rightarrow$).

Pentru tine, ca viitor QA Automation Engineer, aceasta este **cea mai importantă sintaxă din acest modul**. Playwright, Cypress și toate framework-urile moderne de testare sunt scrise bazându-se aproape exclusiv pe Arrow Functions.

Cum transformăm funcția clasică de mai sus într-o funcție săgeată? Scoatem cuvântul function, adăugăm o săgeată $\Rightarrow$ între parametri și acolade, și o stocăm frumos într-o constantă:

```
const calculeazaPretTotal = (pretBaza, taxa) => {
  let pretFinal = pretBaza + taxa;
  return pretFinal;
};

console.log(calculeazaPretTotal(50, 5)); // Afișează: 55
```

De ce le iubim în Playwright? Când vei ajunge la Modulul 4, absolut fiecare test pe care îl vei scrie va arăta fix așa:

```
test("Verifică butonul de Login", async ({ page }) => {
  // Aici vine corpul funcției tale de test! Toată logica ta E2E se scrie
```

```
aici.  
});
```

Vezi **săgeata**? Testul în sine este pur și simplu o **Arrow Function** pe care framework-ul Playwright o execută pentru tine!

4.4 Știați că...?



- **Return implicit:** Dacă funcția ta săgeată este extrem de simplă și are o singură linie de cod care doar returnează un rezultat, poți renunța complet la acolade {} și la cuvântul return. Ex: `const dubleaza = (numar) => numar * 2;`. Super scurt și elegant!
- **Funcții fără parametri:** Dacă o funcție nu are nevoie de niciun "ingredient" de intrare pentru a-și face treaba, parantezele rămân goale: Ex: `const saluta = () => { console.log("Salut QA!"); };`
- **Sunt "Cetățeni de rangul 1" (First-class citizens):** În JavaScript, poți face cu funcțiile exact ce faci cu orice altă variabilă sau valoare. Le poți salva, le poți pune în Array-uri sau, mult mai interesant, poți trimite o funcție ca parametru *în interiorul altei funcții* (un concept avansat numit *Callback*, pe care se bazează puternic JavaScript-ul asincron).

4.5 Exerciții Practice

Deschide DevTools (tab-ul Console) în browser sau mediul integrat din VS Code pentru a exersa:



Misiunea 1: Scrie o funcție clasică numită `salutUtilizator` care primește un parametru `nume` (un String) și folosește `console.log` pentru a printa: "Bine ai venit, [nume]!". Apelează funcția dându-i ca parametru numele tău.



Misiunea 2 (Logica de validare): Scrie o **funcție săgeată (arrow function)** numită `verificaAcces` care primește doi parametri: `esteAdmin` (Boolean) și `esteLogat` (Boolean).

- În corpul funcției, folosește un if (structură învățată în capitolul trecut). Dacă ambele condiții sunt true (folosește operatorul logic &&), funcția trebuie să returneze String-ul "Acces permis la Dashboard".
- Altfel (else), să returneze "Acces interzis".
- Apelează funcția în consolă cu valorile true, true și apoi cu false, true, folosind console.log() pentru a vedea rezultatul live.

4.6 Soluții la Exerciții

☑ Soluție Misiunea 1:

```
function salutUtilizator(ume) {
    console.log("Bine ai venit, " + ume + "!");
}

salutUtilizator("Alex");
```

☑ Soluție Misiunea 2:

```
const verificaAcces = (esteAdmin, esteLogat) => {
    if (esteAdmin === true && esteLogat === true) {
        return "Acces permis la Dashboard";
    } else {
        return "Acces interzis";
    }
};

console.log(verificaAcces(true, true)); // Va printa: Acces permis la Dashboard
console.log(verificaAcces(false, true)); // Va printa: Acces interzis
```

4.7 🧠 Mentalitatea de Inginer în Era AI: Locul tău pe

piață

📌 QA vs. AI: Arhitectura Modulară vs. Codul "Spaghetti" (Spaghetti Code)

- **Ce face AI-ul bine:** Asistenții precum GitHub Copilot sau Cursor sunt extrem de rapizi în a genera cod funcțional pentru a îndeplini o sarcină punctuală. Dacă îi spui "*scrie-mi un test pentru achiziția unui produs*", el îți va scrie de sus până jos absolut toți pașii.
- **Capcana (Unde AI-ul eșuează):** Prin natura lor, LLM-urile au tendința de a genera cod "monolitic" și repetitiv dacă nu primesc un context și instrucțiuni arhitecturale stricte. Dacă ai 50 de teste de Checkout generate automat de AI, s-ar putea să te trezești că logica lungă de completare a formularului de livrare (Nume, Adresă, Card) a fost duplicată identic în toate cele 50 de fișiere de test. Dacă mâine dezvoltatorii schimbă selectorul HTML pentru câmpul "Județ", va trebui să cauți și să modifice manual codul în 50 de locuri diferite. Acesta este un cod considerat *unmaintainable* (imposibil de menținut).
- **Cum gândește un inginer QA:** Inginerul se comportă ca un Arhitect. El citește scenariul generat de AI și observă imediat repetiția periculoasă. Va lua logica de completare a adresei și o va extrage într-o **Funcție Săgeată** separată (ex: `completeazaAdresa = (oras, judet) => { ... }`) pe care o va exporta și o va apela ca o comandă simplă în cele 50 de teste. Acum, dacă interfața aplicației se modifică, inginerul actualizează selectorul *într-un singur loc* (în corpul funcției sale), iar toate cele 50 de teste sunt "reparate" instantaneu!
- **De ce vei rămâne relevant pe piață:** Valoarea ta comercială uriașă pentru o companie derivă din capacitatea de a construi **Framework-uri scalabile**. Gândirea în funcții, module și clase (ca viitorul *Page Object Model* pe care îl vom învăța la Modulul 6) este o viziune strategică pe care AI-ul nu o poate planifica de unul singur la nivel macro, pentru o aplicație de tip enterprise aflată în continuă dezvoltare. Tu nu ești doar un "scriitor de comenzi automatizate", tu asiguri sustenabilitatea codului pe termen lung, reducând drastic timpul de mentenanță al întregii echipe.

Capitolul 5: Tema Sesiunii 3 & Extra Practice (Nivel Avansat)

5.1 Tema pentru Acasă (Proiect Sesiunea 3)

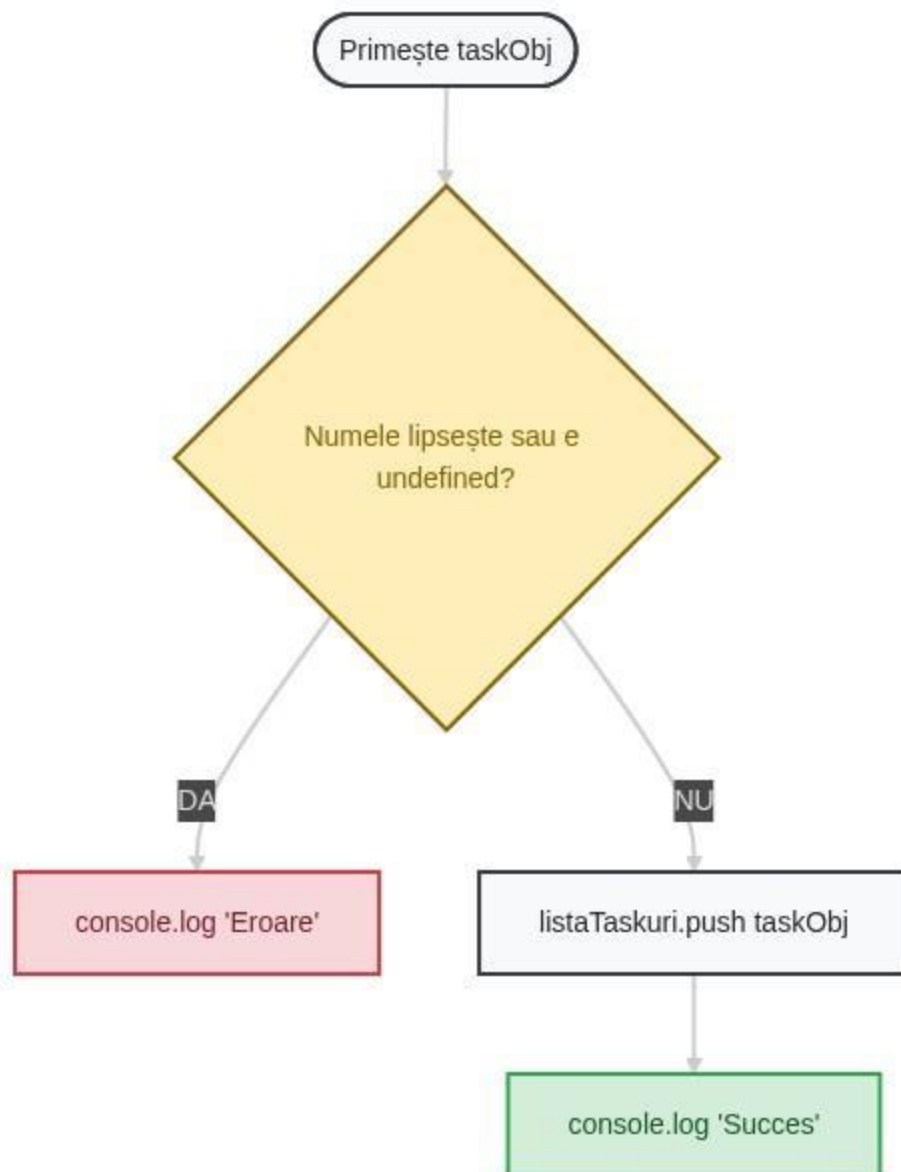


Fig. 5 - Diagrama vizualizează criteriile de acceptare (**Acceptance Criteria**) pentru funcția de validare din cadrul temei:

- **Pornire:** Funcția începe prin primirea unui obiect ca argument: taskObj.
- **Condiția de validare:** Se evaluează structura obiectului printr-un bloc de decizie galben: „Numele lipsește sau e undefined?”.
- **Cazul de eroare (DA):** Dacă proprietatea lipsește, codul urmează ramura din stânga și execută console.log 'Eroare' (bloc roșu), oprind procesarea.
- **Cazul de succes (NU):** Dacă obiectul este valid, urmează ramura din dreapta:
 - Mai întâi, adaugă obiectul în colecție folosind metoda listaTaskuri.push

taskObj.

- Imediat după, confirmă acțiunea prin console.log 'Succes' (bloc verde).

Sarcina ta (Task-ul de business):

Până acum ai construit "scheletul" (**Sesiunea 1**) și "designul" (**Sesiunea 2**) aplicației noastre "Task Tracker". Acum este momentul să îi dăm viață construindu-i prima parte din "creier" (logica de bază). Trebuie să scrii un script **JavaScript** care stochează și procesează lista de task-uri, validând datele *înainte* de a fi afișate pe ecran.

Notă pentru execuție: Vei scrie acest cod fie într-un fișier separat **app.js** (și îl vei lega în HTML-ul tău cu `<script src="app.js"></script>` la finalul `<body>`), fie direct în tab-ul Console din Chrome DevTools pentru testare rapidă.

Cerințe Tehnice (Acceptance Criteria pentru validare):

-  **Structura Datelor E2E (Arrays & Objects):**
 - Declară o constantă numită **listaTaskuri** care să fie un Array.
 - Inițializează acest Array cu cele două task-uri existente deja în tabelul tău HTML din Sesiunile trecute. Fiecare task trebuie să fie un Obiect cu 3 proprietăți: **id** (Number), **nume** (String) și **completat** (Boolean).
 - **Exemplu:** [{ id: 1001, nume: "Setare Boilerplate HTML", completat: true }, ...]
-  **Funcția Săgeată de Validare (Business Logic):**
 - Scrie o **Arrow Function** numită **procesezaTaskNou**. Aceasta primește un singur parametru: **taskObj** (un Obiect).
 - **Validare strictă (if/else):** Verifică dacă numele task-ului lipsește ("") SAU este undefined.
 - Dacă datele sunt invalide, folosește console.log pentru a afișa: **"Eroare: Task-ul trebuie să aibă un nume!"**.
 - Dacă datele sunt corecte, adaugă noul obiect în Array-ul **listaTaskuri** (folosind metoda push()) și afișează: **"Task adăugat cu succes!"**.
-  **Bucula de Raportare (Repetiția):**
 - După ce ai testat funcția de mai sus (adăugând manual un task valid, de ex: id: 1003), scrie o buclă for.

- Această buclă trebuie să parcurgă întregul Array listaTaskuri și să folosească console.log pentru a lista în consolă doar *numele* fiecărui task existent în memorie la acel moment.

5.2 Provocarea Bonus (Opțional): Capcana "Falsy" a API-urilor

Scenariul:

Aplicația ta Task Tracker este pe producție. Primești de la server un răspuns API cu un cod de status. Trebuie să scrii o funcție simplă de logare: dacă statusul e 200, printezi "OK", altfel printezi "Eroare Server".

Un coleg scrie acest cod:

```
const verificaServer = (statusCode) => {
  if (!statusCode) { // Se bazează pe conceptul de Falsy
    console.log("Eroare Server sau Lipsă Răspuns");
  } else {
    console.log("Status OK: " + statusCode);
  }
};
```

Testul tău rulează și dă *False Positive* (trece deși e stricat). Într-o zi, serverul returnează un cod valid pe care colegul nu l-a prevăzut (de ex, codul 0 pentru starea de "așteptare" a unui serviciu intern). Deoarece 0 este evaluat direct ca *Falsy* în JS, funcția colegului dă eroare, deși serverul funcționează!

Misiunea ta:

Cum rescrii acel bloc if/else aplicând o **verificare strictă de tip (=== și typeof)**? Funcția ta, numită verificaServerStrict(statusCode), trebuie să returneze "Status OK" DOAR dacă parametrul primit este de tipul Number. Dacă primește null, undefined sau orice String (ex: "200"), trebuie să returneze "Eroare format date server".

5.3 Cheile de Rezolvare (Soluții & Explicații)

❌ 🚫 🚧 Nu trișa! Citește asta doar după ce ai încercat să scrii singur codul.

☑ Soluție Tema pentru Acasă:

```
// 1. Array de Obiecte (Baza noastră de date temporară)
const listaTaskuri = [
  { id: 1001, nume: "Setare Boilerplate HTML", completat: true },
  { id: 1002, nume: "Înțelegere Arhitectură Client-Server", completat: false }
];

// 2. Funcția de Validare (Arrow Function)
const proceseazaTaskNou = (taskObj) => {
  // Folosim || (SAU logic) și === (egalitate strictă)
  if (taskObj.nume === "" || taskObj.nume === undefined) {
    console.log("Eroare: Task-ul trebuie să aibă un nume!");
  } else {
    listaTaskuri.push(taskObj);
    console.log("Task adăugat cu succes!");
  }
};

// Simulăm acțiunea utilizatorului adăugând un task greșit și unul corect:
proceseazaTaskNou({ id: 1003, nume: "", completat: false }); // Printează: Eroare
proceseazaTaskNou({ id: 1004, nume: "Scrie primul test Playwright", completat: false }); // Printează: Succes

// 3. Bucla de raportare (Lista acum are 3 elemente valide)
console.log("--- Raport Task-uri ---");
for (let i = 0; i < listaTaskuri.length; i++) {
  console.log("Task-ul " + listaTaskuri[i].id + ": " + listaTaskuri[i].nume);
}
```

☑ Soluție Provocarea Bonus (Verificarea Strictă):

```
const verificaServerStrict = (statusCode) => {
  // În loc să ne bazăm pe Falsy, validăm clar contractul de date:
  // Trebuie să fie Număr!
  if (typeof statusCode === "number") {
    return "Status OK: " + statusCode;
  }
}
```

```
    } else {  
        return "Eroare format date server";  
    }  
};  
  
console.log(verificaServerStrict(0)); // Returnează "Status OK: 0"  
(Am reparat bug-ul colegului!)  
console.log(verificaServerStrict("200")); // Returnează "Eroare format  
date server" (Stringurile nu sunt permise)
```

🐛 **Felicitări!** Integrând acest script în **app.js** și legându-l la HTML-ul tău din Sesiunile 1 și 2, tocmai ai pus bazele primei tale aplicații funcționale (Frontend-Logic). Când vom trece la Playwright, roboții tăi vor interacționa cu DOM-ul pentru a declanșa exact aceste funcții de verificare strictă pe care le-ai scris astăzi!